

React.js Interview Questions & Answers – Beginner/Fresher Level

Q1. What is React.js?

React.js is an open-source, front-end JavaScript library for building user interfaces (UIs) based on UI components. It's maintained by Facebook and a community of developers. It is declarative, efficient, and flexible.

Q2. What are the key features of React?

- Component-Based
- Declarative
- Virtual DOM
- Unidirectional Data Flow
- JSX
- Learn Once, Write Anywhere

Q3. What is JSX? Why do we use it?

JSX (JavaScript XML) is a syntax extension for JavaScript. It allows HTML-like code within JavaScript, improving code readability and enabling compile-time optimizations.

Q4. What is the Virtual DOM? How does it work?

Virtual DOM is an in-memory representation of the real DOM. React uses it to detect changes and update only the parts of the DOM that changed, improving performance.

Q5. Difference between Real DOM and Virtual DOM?

- Real DOM: Actual DOM; slow to update.
- Virtual DOM: Lightweight copy; faster updates via diffing and patching.

Q6. What are React Components?

Components are reusable, independent blocks that return elements to render UI. They accept inputs (props) and manage state.

Q7. Difference between Functional and Class Components?

- Functional: Simpler, stateless (or with Hooks), no 'this'.
- Class: More complex, stateful, lifecycle methods, uses 'this'.

Q8. What are Props in React?

Props are read-only data passed from parent to child components to configure them or pass data. Props are immutable.

Q9. What is State in React?

State is internal data managed within a component. It is mutable and triggers re-renders when updated.

Q10. Difference between Props and State?

Feature	Props	State	
-----	-----	-----	
Mutable	No	Yes	
Ownership	Parent component		The component itself
Purpose	Configuration	Internal data	

Q11. How do you pass data between components?

- Parent to Child: via props
- Child to Parent: via callback functions
- Sibling to Sibling: Lift state up to common parent

Q12. What are React Hooks? Why were they introduced?

Hooks are functions to use state and lifecycle in functional components. Introduced in React 16.8 to avoid classes and promote reusable logic.

Q13. Common React Hooks?

- useState
- useEffect
- useContext
- useRef

Q14. Explain useState() Hook.

Allows state in functional components. Example:
`const [count, setCount] = useState(0);`

Q15. Explain useEffect() Hook.

Runs side effects after render. Used for data fetching, timers, and subscriptions.

Q16. What is a dependency array in useEffect?

Second argument to useEffect to control when it runs.

`[]` = only once, `[x]` = on x change, `none` = after every render

Q17. What is conditional rendering?

Render components based on conditions using if, ternary (`?`), or `&&` operators.

Q18. What are Lists and Keys in React?

Lists are rendered using `array.map()`. Keys are unique IDs to help React track changes efficiently.

Q19. What are Controlled Components?

Form elements controlled by React state. Their value is set via state and updated with onChange.

Q20. What are Uncontrolled Components?

Form elements managed by the DOM using refs. Less common in React compared to controlled components.

Q21. What is Prop Drilling? How to avoid it?

Passing props through many levels unnecessarily. Avoid using Context API or state management libraries.

Q22. What is React Router?

Library for handling navigation and routing in SPAs. Lets you map URLs to components without reloading the page.

Q23. Purpose of render() in Class Components?

The render method returns JSX and is the only required method in a class component.

Q24. How to handle events in React?

Use camelCase (onClick), pass function handlers, and call event.preventDefault() for custom behavior.

Q25. Significance of 'key' prop in lists?

Used to uniquely identify items in a list and optimize rendering.

Q26. How to optimize functional components?

- React.memo
- useCallback
- useMemo
- Avoid unnecessary state updates

Q27. What is a Higher-Order Component (HOC)?

A function that takes a component and returns a new enhanced component.

Q28. When would you use a HOC?

To reuse logic like auth, styling, or data fetching across multiple components.

Q29. What is Context API?

A way to share data globally across components without passing props manually through every level.

Q30. When to use Context API?

Global data like user, theme, language. Not ideal for complex state.

Q31. What is the purpose of StrictMode in React?

Tool for highlighting potential issues during development (e.g., deprecated APIs, side effects).

Q32. What is Lifting State Up?

Move state to a common parent when multiple components need to share it.

Q33. What are Fragments in React?

Allow grouping multiple elements without adding extra DOM nodes. Shorthand: `<>...</>`

Q34. Purpose of ReactDOM.render()?

Mounts the React application into the real DOM. Example:
`ReactDOM.render(<App />, document.getElementById('root'))`

Q35. Difference between npm and npx?

- npm: Installs packages.
- npx: Runs package binaries without installing globally.

Q36. What is useRef() in React?

useRef() returns a mutable ref object whose .current property persists across renders. Commonly used to reference DOM elements or store values without triggering re-renders.

Q37. What is useMemo() in React?

useMemo() is used to memoize expensive computations so that they are not re-calculated unless dependencies change, improving performance.

Q38. What is useCallback() in React?

useCallback() memoizes a function so it doesn't get re-created on every render, helpful in optimizing child component renders.

Q39. What is React.StrictMode and how does it help?

A wrapper component that helps identify potential issues in a React app. It runs additional checks and warnings for its children in development mode.

Q40. How do you handle forms in React?

Use controlled components where form values are tied to state. Use onChange handlers to update state and onSubmit to manage submission logic

Q41. What is the difference between React and Angular?

- **React:** Library focused on UI, uses virtual DOM, component-based.
- **Angular:** Full-fledged framework, uses real DOM, includes routing, services, etc.

Q42. What is the role of Babel in React?

Babel is a JavaScript compiler that converts JSX and ES6+ code into browser-compatible JavaScript.

Q43. What is Webpack in a React project?

Webpack is a module bundler that compiles JavaScript modules into a single file and helps manage assets like CSS, images, etc.

Q44. What is reconciliation in React?

The process React uses to compare the new Virtual DOM with the previous one and update the real DOM with minimal operations.

Q45. Can we return multiple elements in React without a parent element?

Yes, by using **Fragments**:

```
<>
```

```
<h1>Hello</h1>
```

```
<p>World</p>
```

```
</>
```

Q46. What are default props in React?

Default values for props that are used when the parent component passes no value.

```
Component.defaultProps = { name: "Guest" };
```

Q47. What is the purpose of the key prop in React?

To uniquely identify list items so React can optimize rendering and avoid unnecessary DOM changes.

Q48. What is the significance of React keys not being index-based?

Using indexes as keys can cause issues with state and component identity, especially when list order changes.

Q49. What is the difference between `useEffect` and `componentDidMount()`?

- `useEffect(() => {}, [])` mimics `componentDidMount()` in functional components.
- `componentDidMount()` is a lifecycle method in class components.

Q50. What are synthetic events in React?

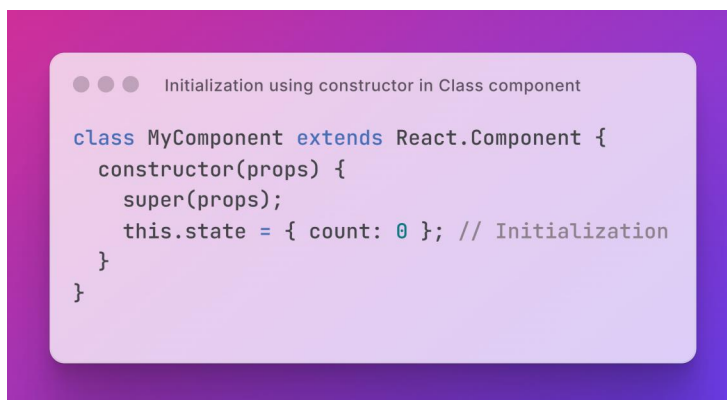
React wraps native events in a SyntheticEvent for cross-browser compatibility and performance.

Q51. What are the different phases of a React component's lifecycle?

There are four different phases in the lifecycle of a React component.

1] initialization

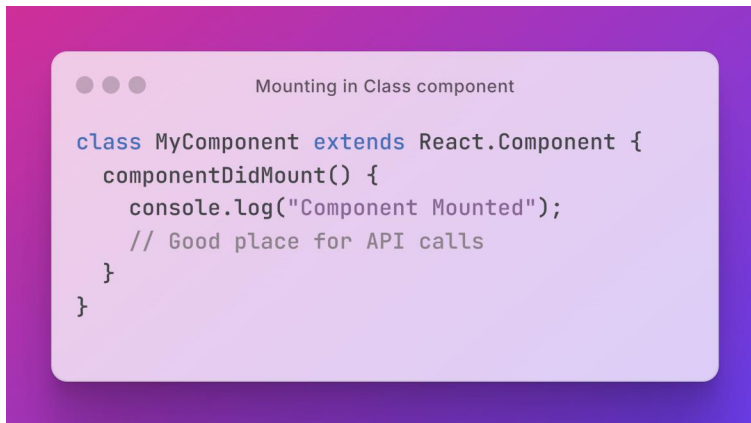
- ✂ What Happens: Component is getting ready.
- ✂ Purpose: Set up initial state and props.
- ✂ Used in: constructor() method (Class Component).



2] Mounting

- ✂ What Happens: Component is inserted into the DOM.
- ✂ Class Methods:

- **constructor()** – Set initial state and props.
- **componentDidMount()** – Runs after the component mounts.



The screenshot shows a code editor window titled "Mounting in Class component". It contains the following JavaScript code:

```
class MyComponent extends React.Component {
  componentDidMount() {
    console.log("Component Mounted");
    // Good place for API calls
  }
}
```

✂ Functional Component Equivalent (Hooks):



The screenshot shows a code editor window titled "Life cycle in Functional component". It contains the following JavaScript code:

```
import React, { useEffect } from 'react';

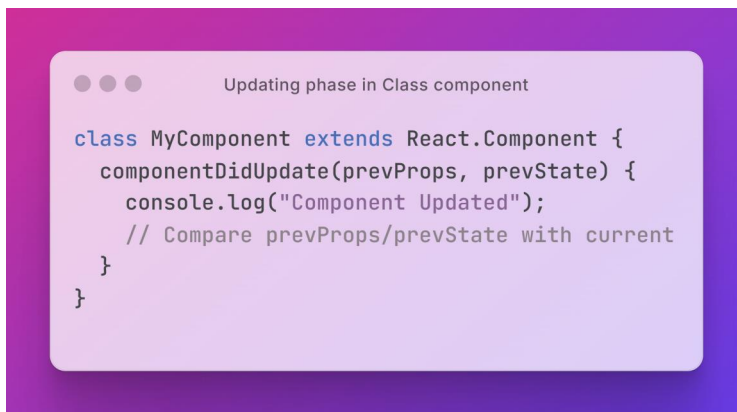
function MyComponent() {
  useEffect(() => {
    console.log("Component Mounted");

    // Cleanup function (like componentWillUnmount)
    return () => {
      console.log("Component Unmounted");
    };
  }, []); // Empty array means this runs only once (mount/unmount)
}
```

3] Updating

- ✂ What Happens: Component re-renders when state or props change.
- ✂ Class Methods:

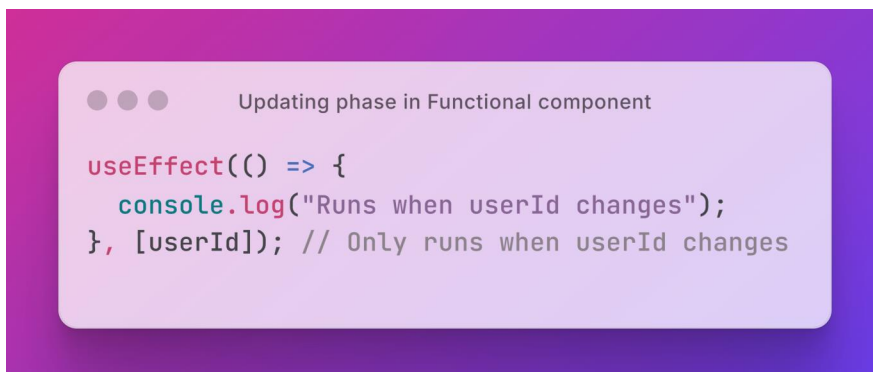
- **shouldComponentUpdate()** – Prevent unnecessary re-renders.
- **componentDidUpdate()** – Runs after re-render due to state/props change.



Updating phase in Class component

```
class MyComponent extends React.Component {  
  componentDidUpdate(prevProps, prevState) {  
    console.log("Component Updated");  
    // Compare prevProps/prevState with current  
  }  
}
```

📌 Functional Component Equivalent (Hooks):



Updating phase in Functional component

```
useEffect(() => {  
  console.log("Runs when userId changes");  
}, [userId]); // Only runs when userId changes
```

4] Unmounting

- 📌 **What Happens: Component is removed from the DOM.**
- 📌 **Class Method:**
 - **componentWillUnmount()** – Cleanup tasks

```
Unmounting phase in Class component

class MyComponent extends React.Component {
  componentWillUnmount() {
    console.log("Component will be removed");
    // Cleanup logic like clearing intervals
  }
}
```

🚀 Functional Component Equivalent (Hooks):

```
Unmounting phase in Functional component

useEffect(() => {
  const interval = setInterval(() => {
    console.log("Running Timer...");
  }, 1000);

  return () => {
    clearInterval(interval);
    console.log("Cleaning up...");
  };
}, []);
```

*This document was created and shared by **Brijisha Doshi** to support the React developer community. Feel free to contribute, update, and re-share it with your additions.*

Let's grow together. 🚀